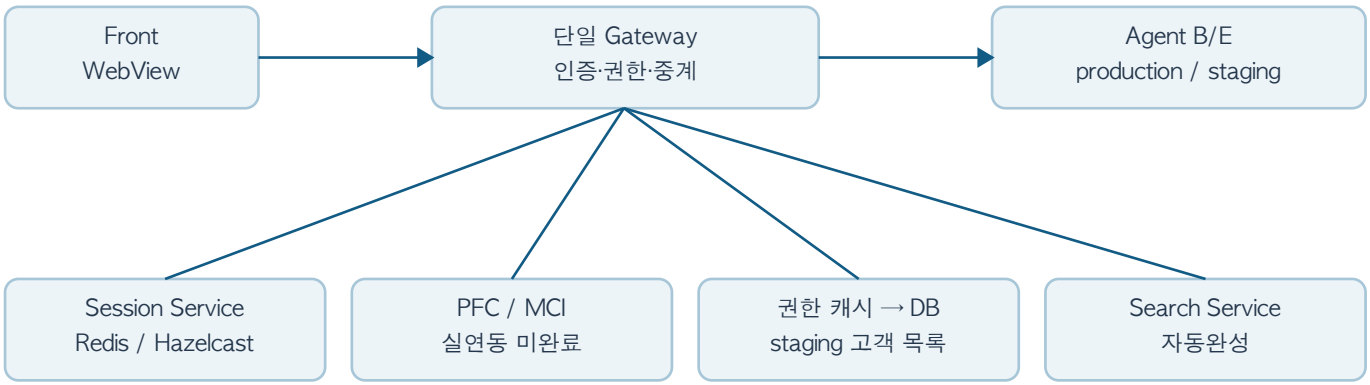
01 프로젝트 개요와 책임

Chatbot Gateway 인수인계서 | 기준일 2026-10-21 | 현재 코드 기준

프론트와 Agent B/E 사이에서 세션 인증, 채널 접근 권한 검사, 추가 인증 검증, HTTP/SSE 중계를 수행한다. 단일 GW가 인증된 세션의 channelId에 따라 staging 또는 production Agent B/E를 선택한다.

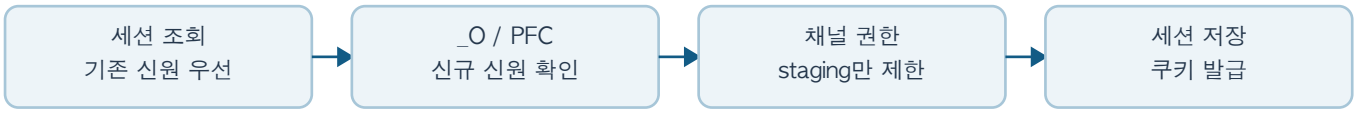


구성	담당 책임
Gateway	Java 21 / Spring Boot 3.5.16 / MVC·Tomcat / Spring Cloud Gateway Server Web MVC. 인증 및 중계 담당.
Agent B/E	대화 이력·thread 소유권, 게스트 이력 연결, 금융 업무 실행 및 중복 실행 방지.
agent-session-service	세션 및 인증 문맥 저장, 교체·연장·인증 소비의 원자 처리. Redis/Hazelcast는 이 서비스 내부 책임.
PFC / MCI	로그인 신원 검증, 추가 인증, 홈페이지 연동 쿠키. 실제 운영 구현은 미완료.
Document DB / Search	DB는 staging 허용 고객 원본. 자동완성은 별도 search-autocomplete-service 호출.

인수인계 핵심: GW에는 대화 이력·업무 실행 상태를 저장하지 않는다. local mock의 성공을 실제 내부 시스템 연동 완료로 설명하지 않는다.

02 세션과 로그인 흐름

세션 쿠키는 __Host-sh-session이며 HttpOnly·Secure·SameSite=Lax·Path=/를 사용한다. sessionId는 인증 식별자, threadId는 대화 식별자다. threadId만으로 인증을 복구하지 않는다.

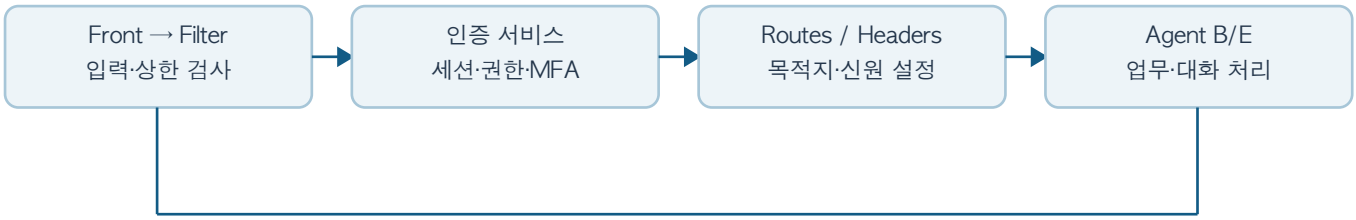


기존 게스트 회원 전환은 chat + X-Requested-Login: true에서만 수행

상황	처리 규칙
최초 진입	POST /api/auth/session?channelId=… . 기존 유효 세션 우선. 신규 생성 시 _O가 있으면 PFC 검증, 없으면 게스트. staging은 게스트 금지.
게스트 로그인 전환	POST /api/chat에 X-Requested-Login: true와 _O 쿠키. PFC PID 확인 후 기존 게스트 세션을 원자 교체. thread·채널·이전 게스트 신원 유지.
전환하지 않는 경우	헤더 생략/false 또는 /api/auth/session에서 기존 게스트 재사용. 유효 회원은 기존 인증을 사용.
새 대화	유효 세션에서 POST /api/auth/session/restart + 새 threadId. 회원 신원 승계, 게스트 식별자는 신규 생성. _O 재발급 불필요.
만료·쿠키 유실	무활동 5분이면 기존 대화 요청 401. restart도 유효 신원이 필요. 새 인증/신규 세션 흐름으로 이동하며 게스트 신원을 threadId로 복원하지 않음.
홈페이지 연동	회원 세션 생성/재사용 및 채팅·도구 요청에서 _I 자동 발급. GET·click·restart는 제외. 발급 실패 시 중계하지 않음.

Idempotency-Key는 새 활동마다 새 값, 네트워크 재전송에는 같은 값을 사용한다. 활동 연장과 restart 중복 처리는 세션 서비스 계약에 의존한다. 금융 업무 중복 실행 방지는 별도로 B/E가 담당한다.

03 대화 중계와 SSE



HTTP / SSE 응답은 공식 Gateway handler로 전달

코드/단계	확인할 동작
ChatGatewayFilter	허용 경로·동시 요청 상한 검사. chat JSON 최대 1MiB, threadId와 MFA 헤더 해석. 본문 원문 보존 및 응답 자원 반환.
ChatAuthenticationService	세션 인증 → 필요한 회원/thread 인증 문맥 저장 → MFA 결과 검증. 업무 실행을 직접 수행하지 않음.
SessionAuthentication	채널·thread·세션 검증. staging 권한 확인. 명시적 로그인 전환 또는 활동 만료 연장.
ChatGatewayRoutes / GatewayHeaders	세션 채널로 목적지 결정. /api/chat… → /internal/v1/chat… . 외부 신원 헤더 제거 후 검증된 신원 주입.
공식 Gateway handler	Agent 응답의 HTTP/SSE 중계. 이벤트 파싱·재직렬화 및 자체 heartbeat 생성 없음.

SSE를 설명할 때 강조할 내용

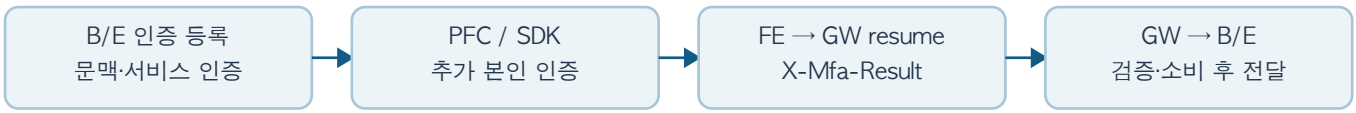
SseEmitter를 직접 사용하지 않아도 공식 Gateway MVC handler가 스트림을 전달한다. Netty 기반 서비스가 아니며, Tomcat과 Java 21 가상 스레드에서 동작한다.

요청 진입 시 인증을 통과했다면 세션 자연 만료로 진행 중인 SSE를 끊지 않는다. 기본 upstream 읽기 간격 제한은 30초이므로 B/E heartbeat 주기와 맞춰야 한다. SSE 커밋 후 오류는 JSON 추가 대신 연결 종료로 처리한다.

기본 동시 중계 상한은 100이며 초과 시 503이다. 운영 성능 보장 수치가 아니라 초기 설정값이다. 연결 풀 반환·브라우저 단절·조건부 응답 처리는 회귀 테스트로 보호한다.

04 API와 추가 인증

인터페이스	역할
POST /api/auth/session	세션 신규 생성 또는 재사용. 기존 게스트 회원 전환은 하지 않음.
POST /api/auth/session/restart	유효 신원으로 새 대화. reset 및 GET 세션 조회 API는 없음.
POST /api/chat	대화·로그인 전환·추가 인증 후 resume, HTTP/SSE 중계.
POST /api/chat/threads/{tid}/tools/{tool}	도구 요청 원문 중계.
GET /api/chat/threads 및 하위 조회	이력·messages·artifacts 조회. 세션 활동 연장 없음.
POST /api/chat/auto	Eureka → search-autocomplete-service /api/autocomplete/chat. 세션 연장 없음.
POST /api/click	세션 연장과 AUDIT JSON 파일 기록. 기존 message/buttonInfo 구조, 일별 분리·14일 보관. B/E 호출 없음. 수집 영구 저장 보장은 아님.
POST /internal/v1/auth/operations	B/E 서비스 인증과 X-Auth-Context를 사용한 추가 인증 요청 등록.



GW는 인증 증명을 검증하고, 실제 금융 업무 실행은 B/E가 담당

MfaResultParser는 X-Mfa-Result(base64url JSON 배열)의 형식·크기·중복을 검사한다. MfaService는 고객·채널·thread·업무 정보의 일치 및 기한을 확인하고, 모든 결과를 검증한 뒤 외부 세션 서비스에 원자 소비를 요청한다.

클릭 로그: sessionId=threadId, userId=검증된 회원 PID(게스트 null). 선택 buttonInfo 5개 문자열과 시나리오·상태를 기록한다. 인증 쿠키는 기록하지 않는다. 파일 경로는 logging.jsonfile.path로 설정한다.

B/E에는 X-Verified-Auth-Requests로 검증된 인증 요청 ID를 전달한다. 이 인증 소비는 금융 업무 실행과 다르며, 실행 여부·실패 복구·중복 실행 방지는 B/E 책임이다.

GW 자체 FE 오류는 주로 400(입력), 401(인증), 403(권한), 503(연동 장애). 404/405/500은 별도 유지. B/E 오류와 내부 API의 상세 계약은 FE용 단순화 규칙과 구분한다.

05 코드를 읽는 순서와 수정 위치

한 기능을 처음부터 끝까지 따라 읽는다. 메소드별로 입력, 검증/변경, 실패 시 응답을 설명하고 세부 HTTP 처리부터 시작하지 않는다. 아래 경로의 기준은 src/main/java/com/example/chatgateway/다.

영역 / 클래스	수정할 때 확인할 책임
controller/ChatSessionController	세션 생성·restart 요청과 응답 DTO.
service/SessionAuthentication	createOrReuse / inspect / authenticateActivity / restart. 세션 수명과 로그인 전환의 중심.
gateway/ChatGatewayFilter	요청 검사·본문 재사용·중계 자원 관리. SSE 종료/오류 관련 수정 시 확인.
service/ChatAuthenticationService	세션·인증 문맥·MFA를 순서대로 연결하는 중계 인증 서비스.
gateway/ChatGatewayRoutes, GatewayHeaders	공개 경로·목적지 선택, 신원 헤더 제거/주입.
service/ChannelAccessService	staging 고객 허용 검사와 Caffeine 캐시.
config/ChannelAccessConfiguration	Mongo 호환 DB 조회와 local 테스트 목록 구성.
service/MfaService, gateway/MfaResultParser	추가 인증 업무 흐름 / HTTP 헤더 파싱.
client/session/HttpSessionServiceClient	세션 API 호출·응답 검증. 저장소를 직접 구현하지 않음.
session/AuthorityStore, LocalAuthorityStore	외부 원자 명령 계약 / local 전용 메모리 구현.
client/pfc/PfcClient, MockPfcClient	운영 PFC 구현 경계 / local 테스트 구현.
service/ClickService, client/search/AutocompleteClient	클릭 로그·활동 연장 / 자동완성 요청·응답 계약.

변경 요청별 빠른 탐색

로그인 정책 → SessionAuthentication. staging 권한 → ChannelAccessService. B/E 주소 → ChatGatewayRoutes와 설정. 세션 API 계약 → AuthorityStore와 HttpSessionServiceClient. SSE 장애 → Filter, GatewayHttpClient, UpstreamClients와 Proxy 테스트.

범용 서비스·전략 계층을 추가하기 전에 기존 책임 경계를 확인한다. 응답 크기 제한, 신원 검증, 원자 교체, 중복 소비 방지는 단순화를 이유로 제거하지 않는다.

06 Staging 권한과 운영 설정

production은 기존 인증 정책을 유지한다. staging은 PFC로 인증한 PID가 Document DB 허용 목록에 있는 회원만 접근한다. 프론트가 보낸 고객번호를 신뢰하거나 목적지 URL을 받지 않는다.

설정/데이터	현재 기준
chat.channels.allowed	허용 채널 ID → PFC. staging 채널도 이 목록에 등록.
chat.staging.channels	staging 채널 ID → Agent B/E HTTP(S) 주소.
chat.python.base-url	staging 외 채널의 production B/E 주소.
chat.staging.cache-ttl / cache-size	기본 1분 / 10,000개. 채널+PID 단위 허용·거절 결과 캐싱.
chat.staging.mongo-uri / database	외부에서 주입할 Mongo 호환 URI / 기본 chat_gateway.
DB 컬렉션	channel_customer_access: channelId, customerId, enabled(boolean). channelId+customerId 고유 인덱스 필요.
캐시 만료/장애	접근 시 TTL을 연장하지 않음. 유효 캐시는 유지, 만료 후 DB 오류는 503. 오류를 허용 결과로 대체하지 않음.

```
chat:
  channels:
    allowed:
      "prod-channel": PFC
      "stage-channel": PFC
  python:
    base-url: https://production-agent.internal
  staging:
    channels:
      "stage-channel": https://staging-agent.internal
    mongo-uri: ${STAGING_DOCUMENTDB_URI}
    database: chat_gateway
    cache-ttl: 1m
    cache-size: 10000
```

위 값은 예시다. 실제 DB 제품·채널 ID·주소는 미확정이다. 현재 어댑터는 MongoDB 호환 기준이며, 실제 제품에 맞는 TLS/연결 옵션 확인이 필요하다. DB 계정은 목록 조회 권한으로 구성하고 데이터·고유 인덱스는 관리 절차로 준비한다.

허용 고객 삭제는 최대 캐시 TTL 후 새 요청에 반영된다. 이미 시작한 SSE는 강제 종료하지 않는다. 목록 관리 API는 없으며 승인된 DB 관리 절차를 사용한다.

07 실행·검증과 인수인계 완료 기준

로컬 실행

```
./gradlew bootRun --args='--spring.profiles.active=local'
./gradlew spotlessApply check bootJar --console=plain
```

Java 21이 필요하다. local은 메모리 세션, MockPfcClient, 설정 기반 허용 고객 목록을 사용한다. 실제 운영에 local 프로필을 사용하지 않는다. B/E는 별도 mock/실행 서버가 있어야 대화 중계를 테스트할 수 있다.

Swagger UI: http://localhost:8080/docs/index.html (local 전용). 포트 충돌 시 실제 실행 포트로 변경한다. HttpOnly/Secure 쿠키의 브라우저 동작은 동일 Origin HTTPS 환경에서 확인한다.

파일/검증	사용 방법
http/session-local.http	세션 생성·로그인 전환·restart 등 기본 시나리오. 환경 baseUrl 확인.
http/staging-local.http	…0002 staging 채널의 게스트 거절 / local-member 허용. 기본 B/E localhost:8002.
http/autocomplete-local.http	검색 서비스 요청/응답. local 주소는 AUTOCOMPLETE_BASE_URL로 지정.
테스트 보고서	build/reports/tests/test/index.html. 클릭 AUDIT 반영 후 전체 148개 통과, 오류·실패·skip 0. 코드 검사·실행 JAR 빌드 통과.
중점 회귀 테스트	SessionAuthenticationTest, MfaServiceTest, StagingAccessHttpTest, ChannelAccessServiceTest, ProxyTransportTest, ProxyDisconnectTest.

운영 인수 전에 남은 사항

항목	인수 담당자가 확인할 것
PFC 실연동	SDK/내부 의존성, MCI 응답 성공 코드, PID·MFA 필드 매핑, _I 쿠키 발급 계약. 현재 운영 기본 구현은 503 차단.
세션 서비스	Eureka 등록, API·서비스 인증 일치, 원자 교체/연장/소비 및 재시도 보장. 실제 연동 미검증.
환경 설정	staging/production 목적지, 서비스 자격증명, Mongo 호환 여부·URI·TLS·허용 목록·고유 인덱스.
운영 E2E	로그인 복귀·iOS 쿠키 유실, SSE 프록시 buffering/timeout, DB·세션 서비스 장애, 로그 수집·보존.

권장 인수 시연: production 게스트 대화 → 로그인 전환 → restart → staging 거절/허용 → SSE → 테스트 보고서. 구현 완료와 외부 연동 완료를 구분하여 기록한다.

상세 참고(저장소 상대 경로): docs/agent-api-gw-session-summary.md, docs/openapi/external-api.yaml, docs/openapi/python-internal-api.yaml. 세션 서비스 계약과 staging·클릭 로그 정책은 기준 설계서에 통합했다.